BFS & DFS in a matrix :-

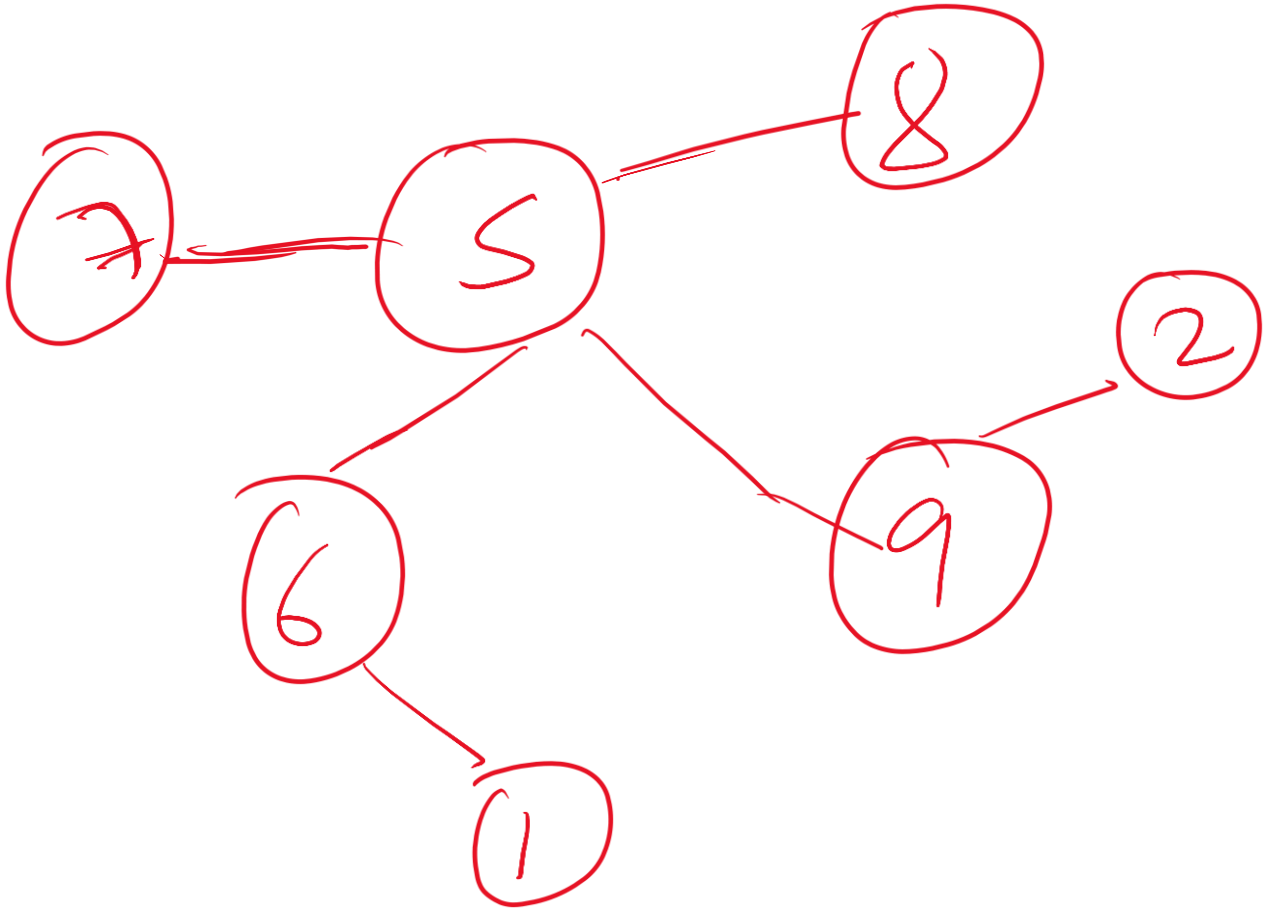
	col 0	col 1	col 2	col 3	col 4
Row → 0	(0, 0)				
Row → 1	(1, 0)	(x-1, y-1)	(x-1, y)	x-1, y+1	
Row → 2	(2, 0)	(x, y-1)	(x, y)	x, y+1	
Row → 3	(3, 0)	x+1, y-1	(x+1, y)	x+1, y+1	

int dx[8] = {-1, -1, 0, 1, 1, 1, 0, -1}

int dy[8] = { 0, 1, 1, 1, 0, -1, -1, -1}

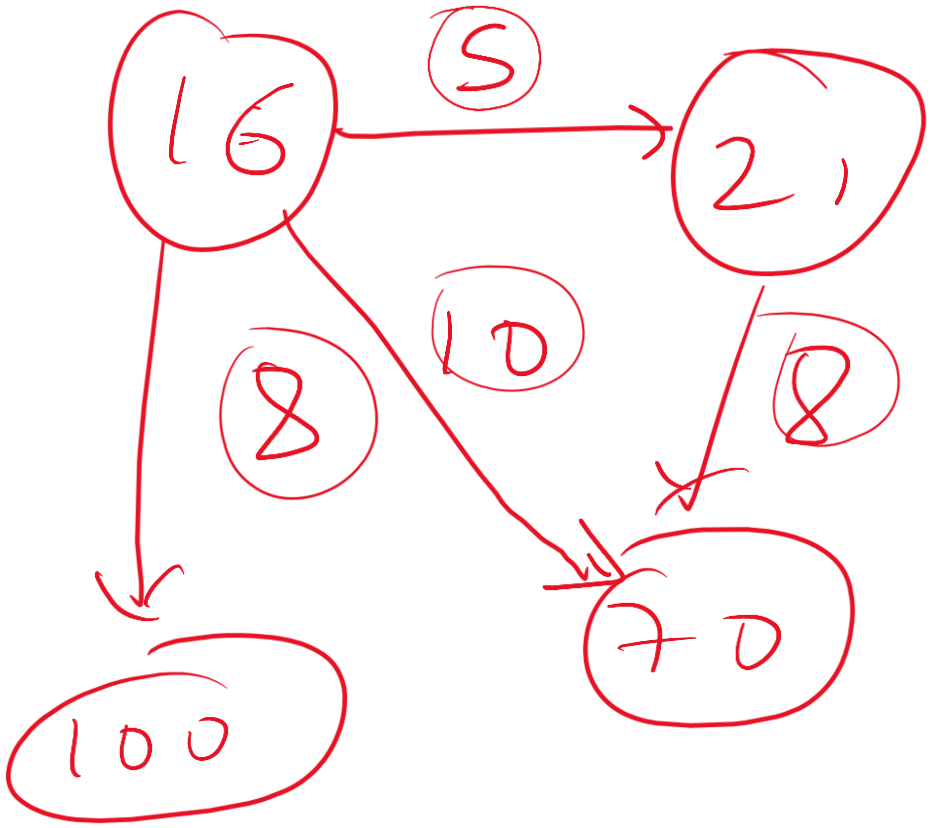
① Unweighted :-

Graph



Weighted :-

Graph



① Matrix is one type of unweighted graph bcoz the cost to reach any of the adjacent cell from current cell will be same.

Whenever in an Interview, you are asked to find out the shortest path from source to destination in a Matrix (or Unweighted Graph), you need to solve the problem using $\boxed{\text{BFS}}$.

```
int dx [ ] = {                    };
int dy [ ] = {                    };

queue.add( ); //Add source point
                  into the queue;
int dist = 0;
while (! q.empty( ))
  {   int n = q.size( );
      dist++;
```

```
while (n--)
{
    int [] point = q.poll();
    int cx = point[0];
    int cy = point[1];
    if(cx == dest.x && cy == dest.y)
    {
        return dist || dist - 1;
    }
```

```
// Traverse adjacent cell
for (int k = 0; k < 8; k++)
{
    int nx = cx + dx[k];
    int ny = cy + dy[k];
    if(nx<0 || ny<0 || nx>=r || ny>=c)
        continue;
    if( vis[nx][ny] == 1)
        continue;
```

```
queue.add(new int[]{nx, ny});
vis[nx][ny] = 1;

}// for k closed

}// while closed

}// while closed.
```

DFS :-

1. DFS in a matrix helps you to find connected component.

1	1	0	0	1	1
0	1	0	0	1	1
0	1	0	1	1	0
0	0	0	1	1	0

Ans = 2

```
void dfs(int x, int y, int r, int c,
        char[][] grid, boolean vis[][])
{ if (x<0 || y<0 || x>=r || y>=c ||
  || grid[x][y]==blocker || vis[x][y]==true)
      return;

  vis[x][y] = true;
```

```
for (int k=0; k< 4; k++)
{ int nx = x + dx[k];
  int ny = y + dy[k];
  if (nx>=0 && ny>=0 &&
      nx < r && ny < r
      && vis[nx][ny]==false)
      dfs(nx, ny, r, c, grid, vis);
}
```

} // dfs for closed